Reinforcement Learning From Human Feedback

Divyam Madaan

November 14, 2024

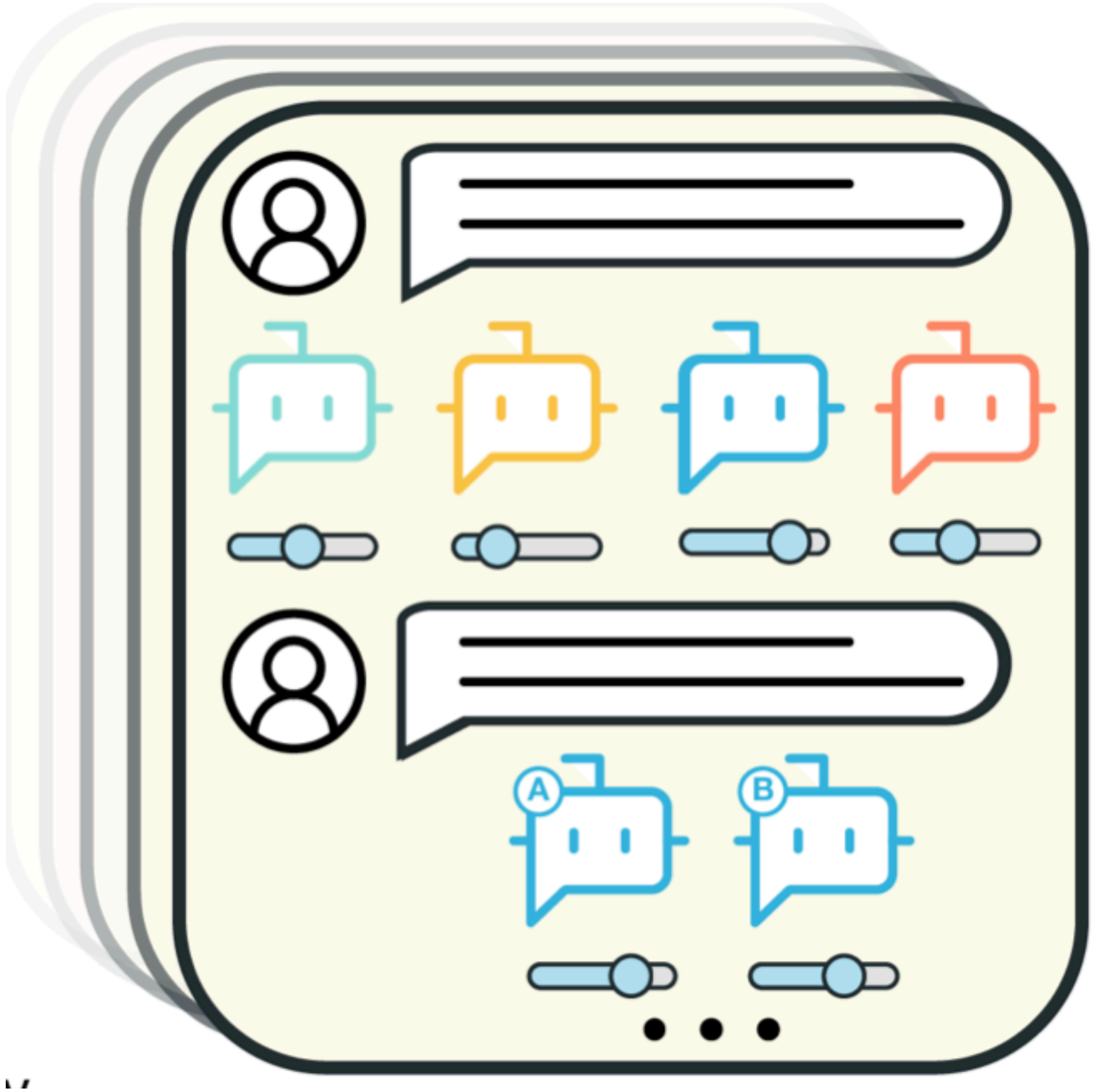
Pictures from https://web.stanford.edu/class/cs224n/slides/cs224n-spr2024-lecture10-prompting-rlhf.pdf

Overview of RLHF

- Pipeline for RLHF
- Step 1: Supervised fine-tuning
- Step 2: How to model human preferences?
- Step 3: Optimizing the reward model

Human-AI Alignment

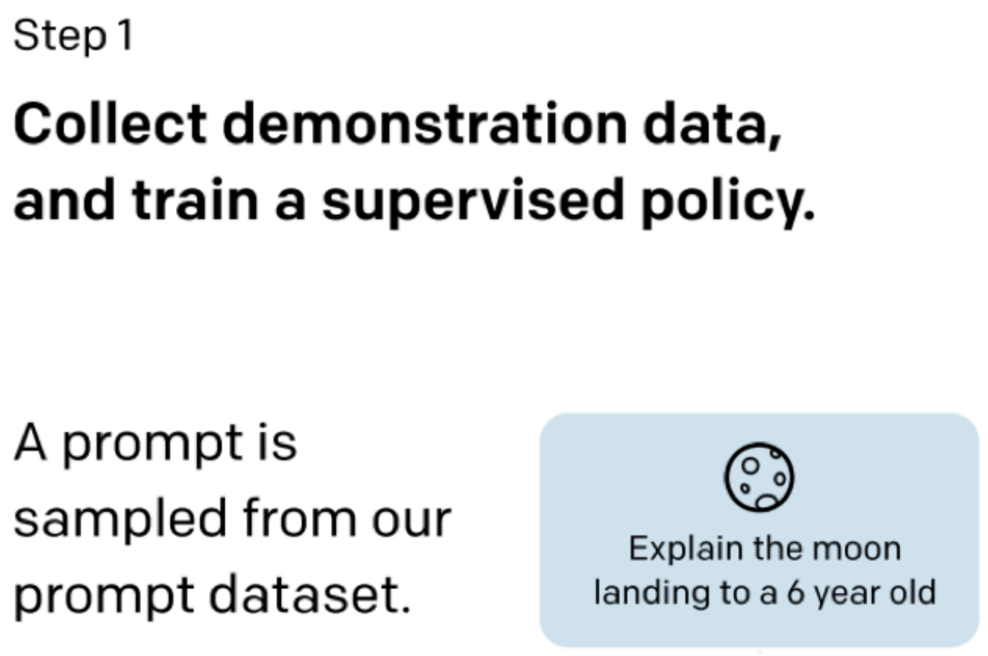
________________ towards human preferences



PRISM Alignment project (Kirk et al., 2024)

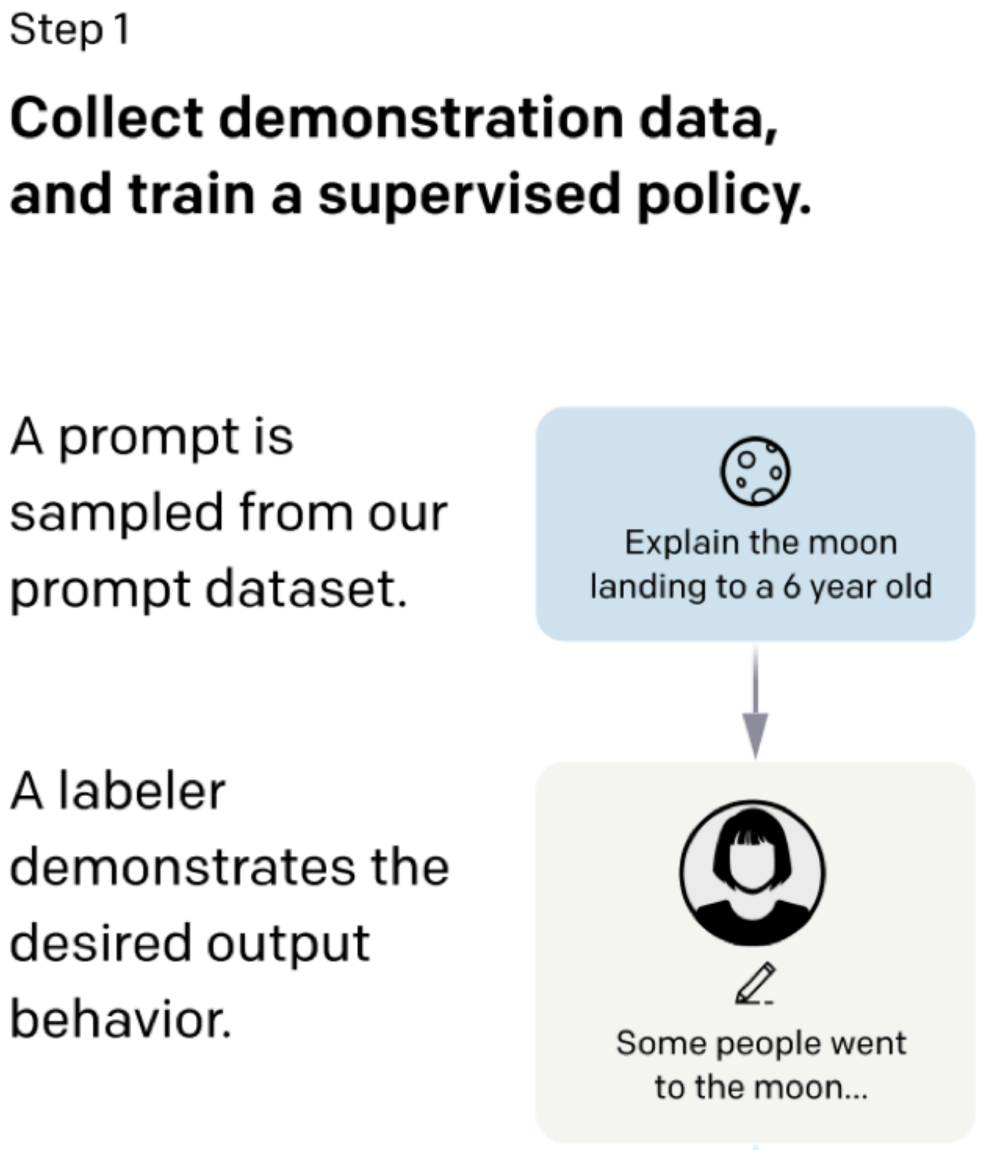
RLHF pipeline

Step 1: Supervised fine-tuning



RLHF pipeline

Step 1: Supervised fine-tuning



RLHF pipeline

Step 1: Supervised fine-tuning

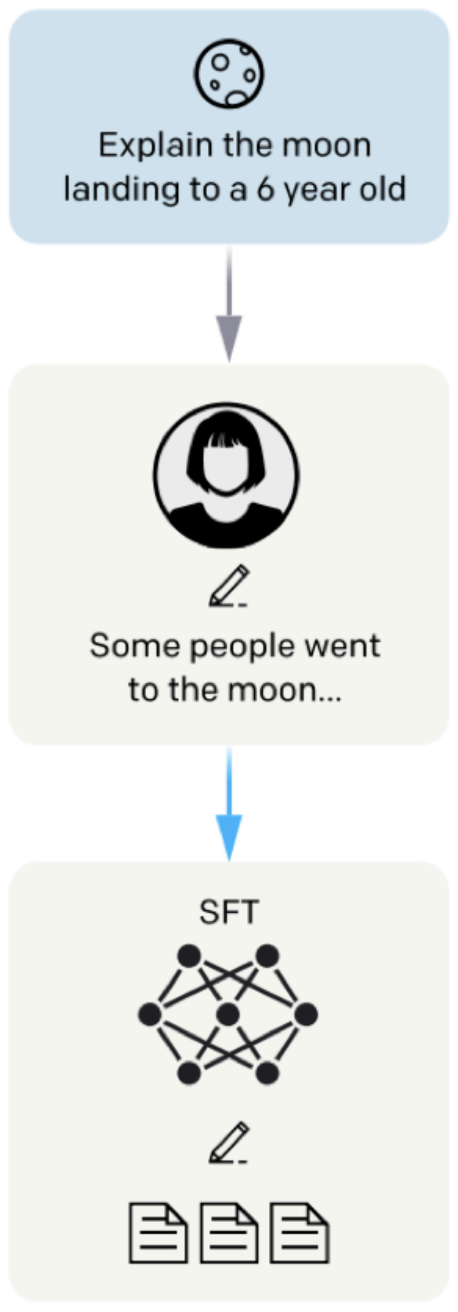
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

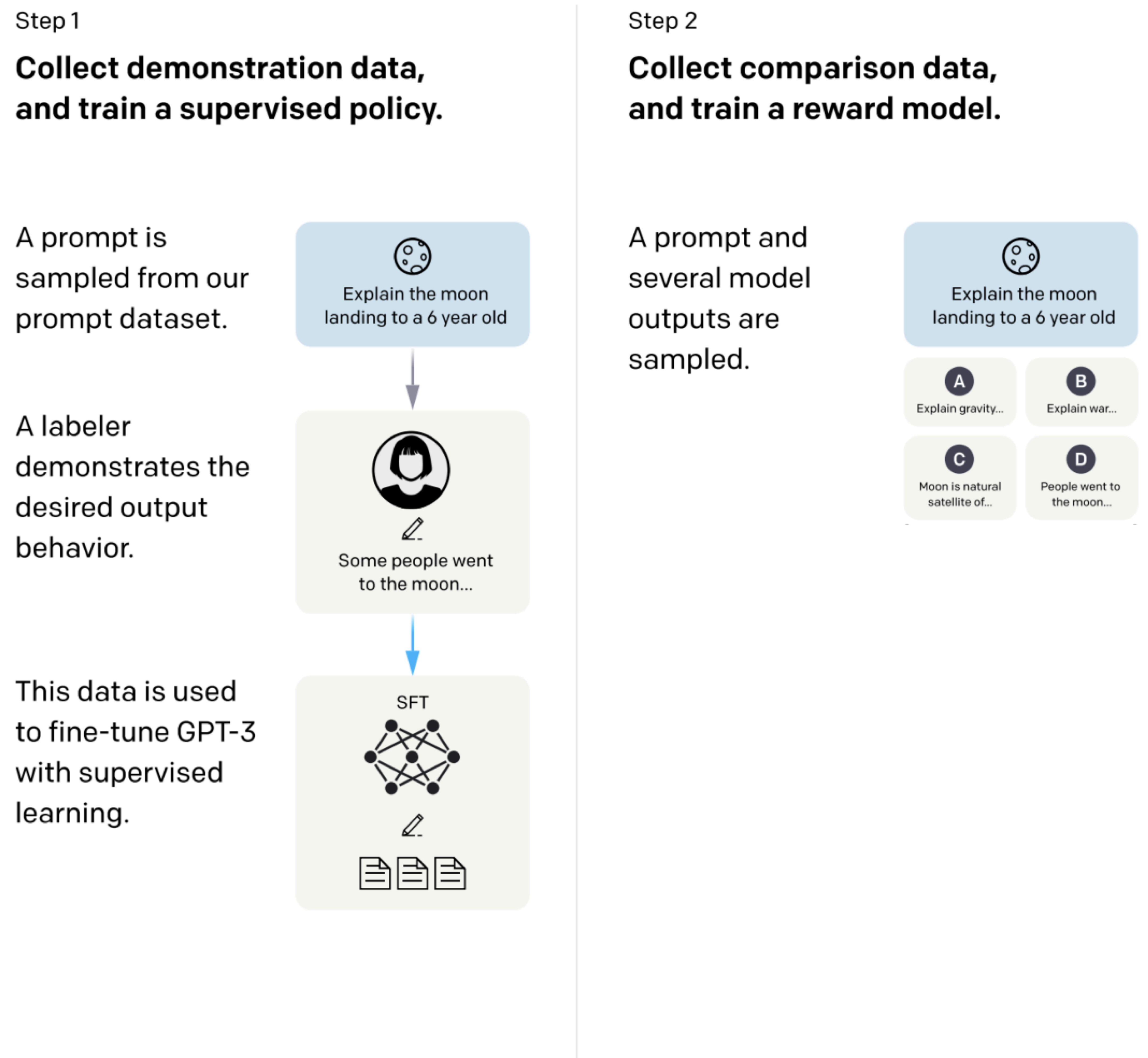
A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



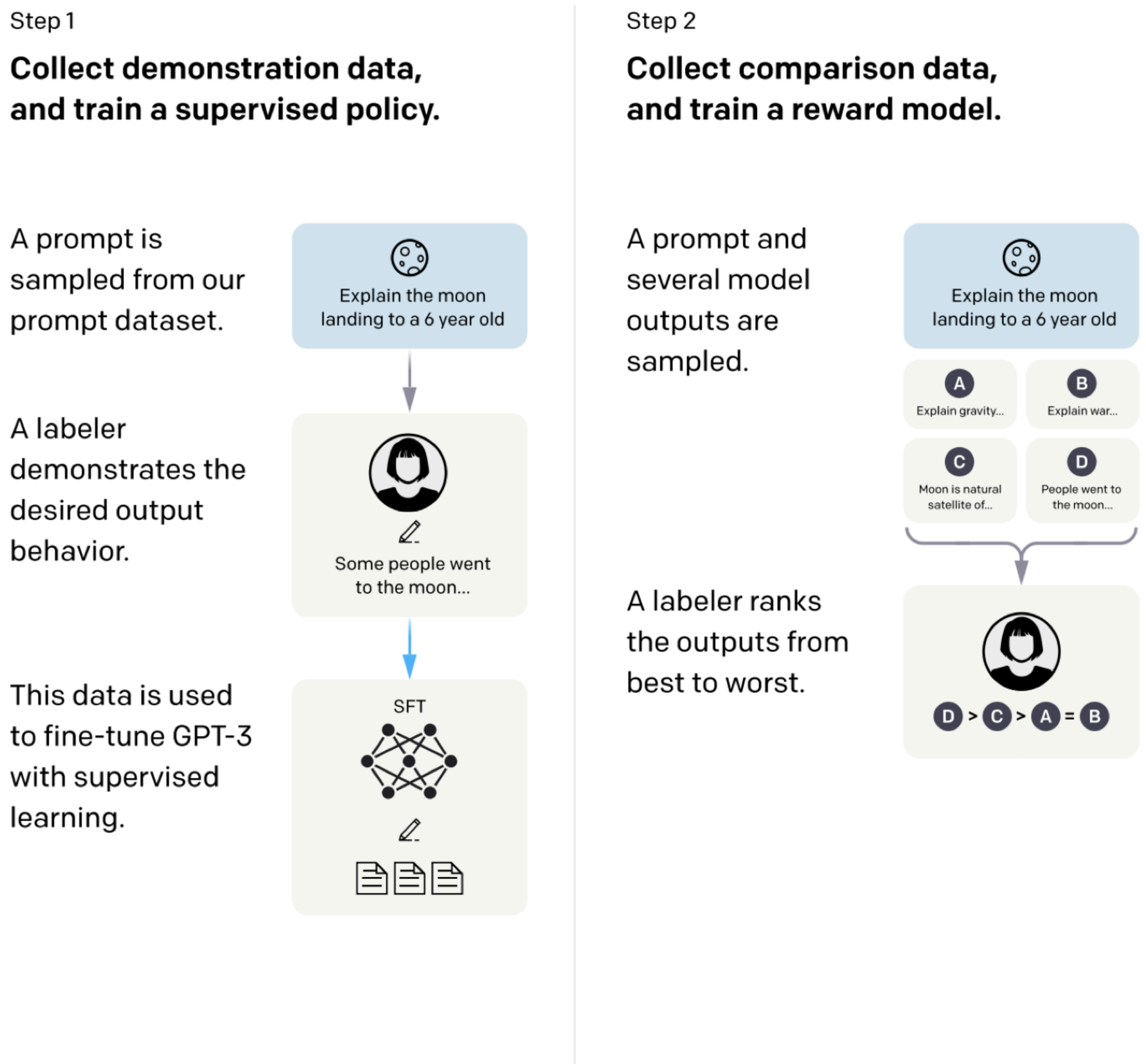
RLHF pipeline

Step 2: Reward model training



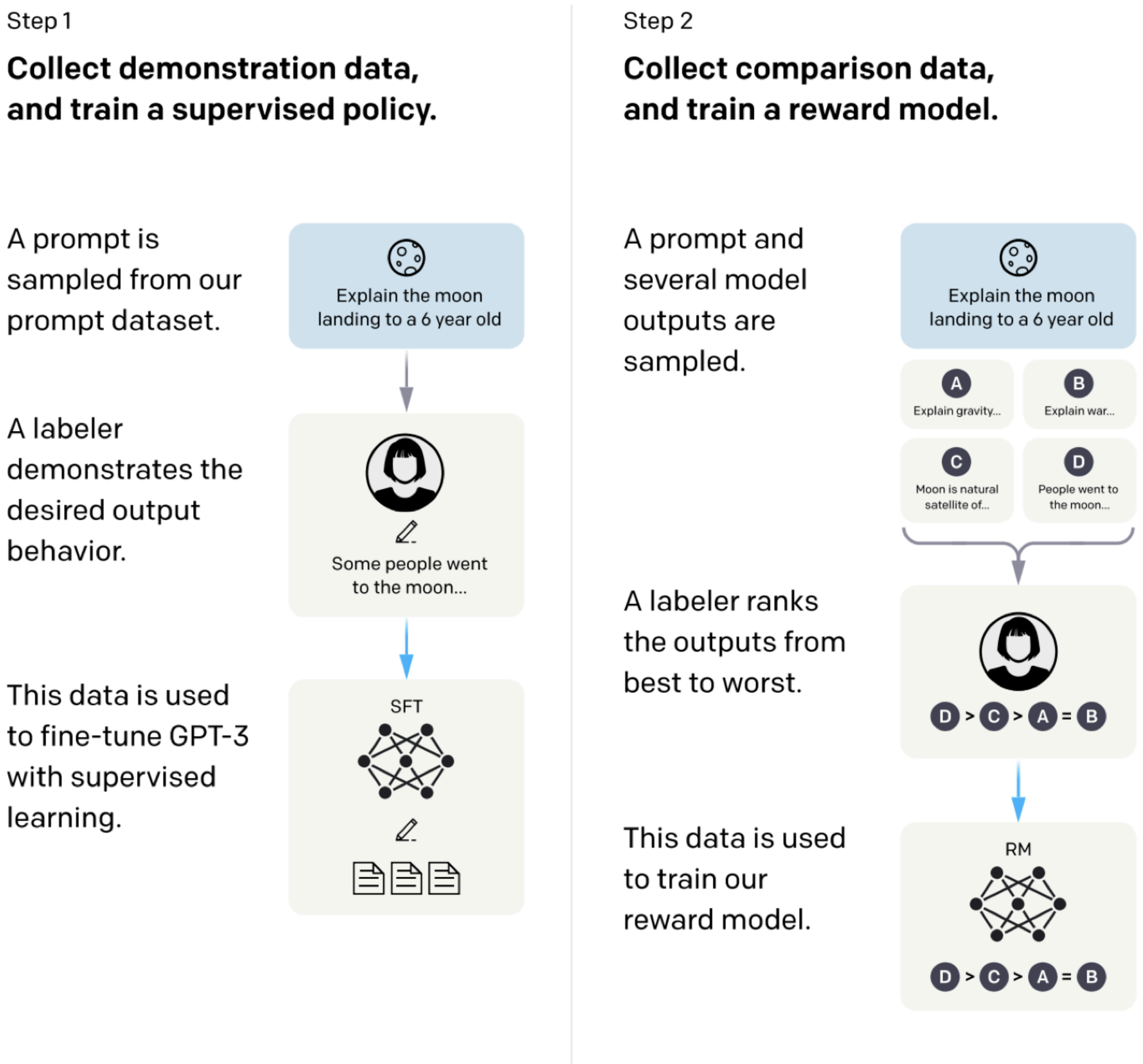
RLHF pipeline

Step 2: Reward model training



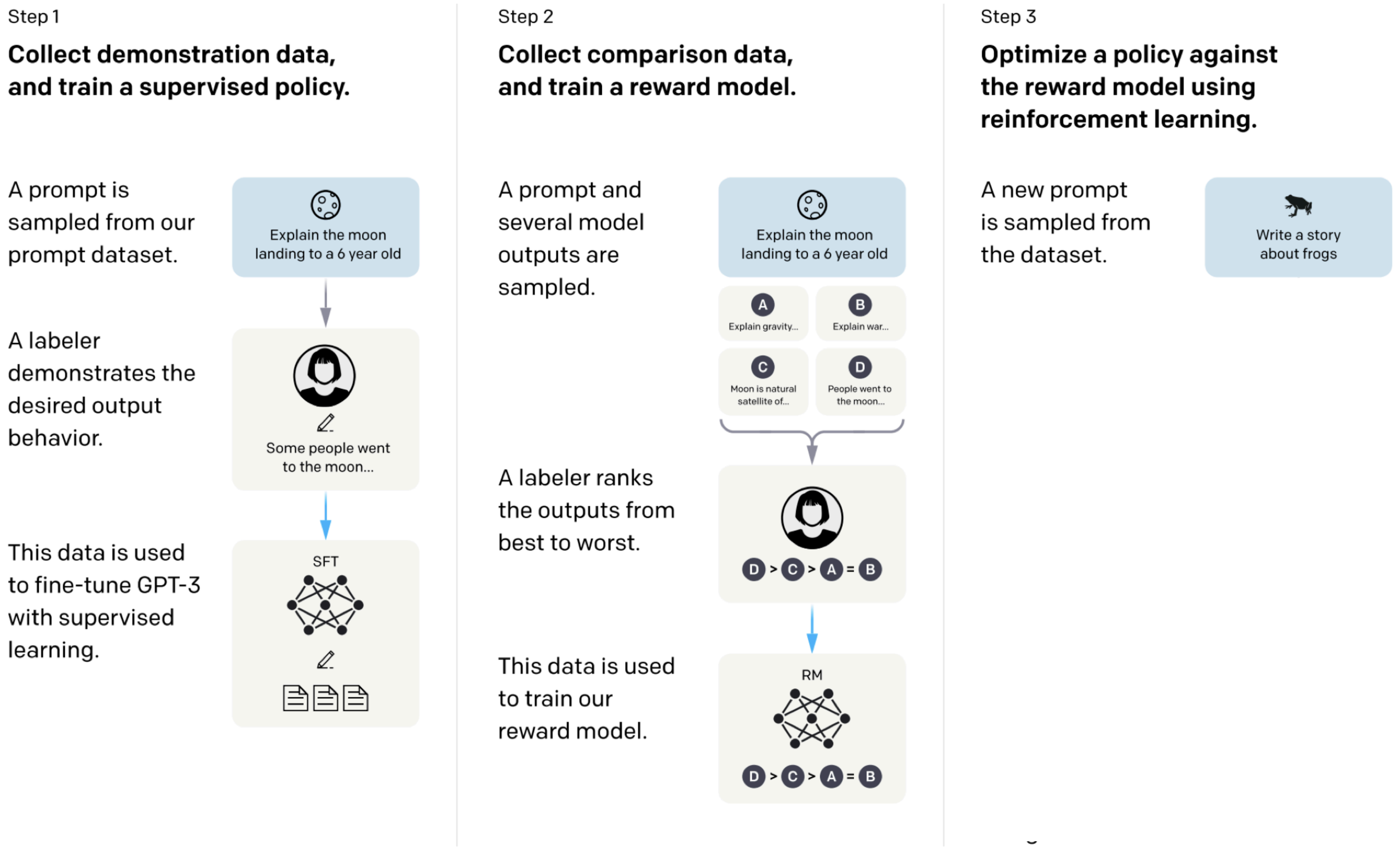
RLHF pipeline

Step 2: Reward model training



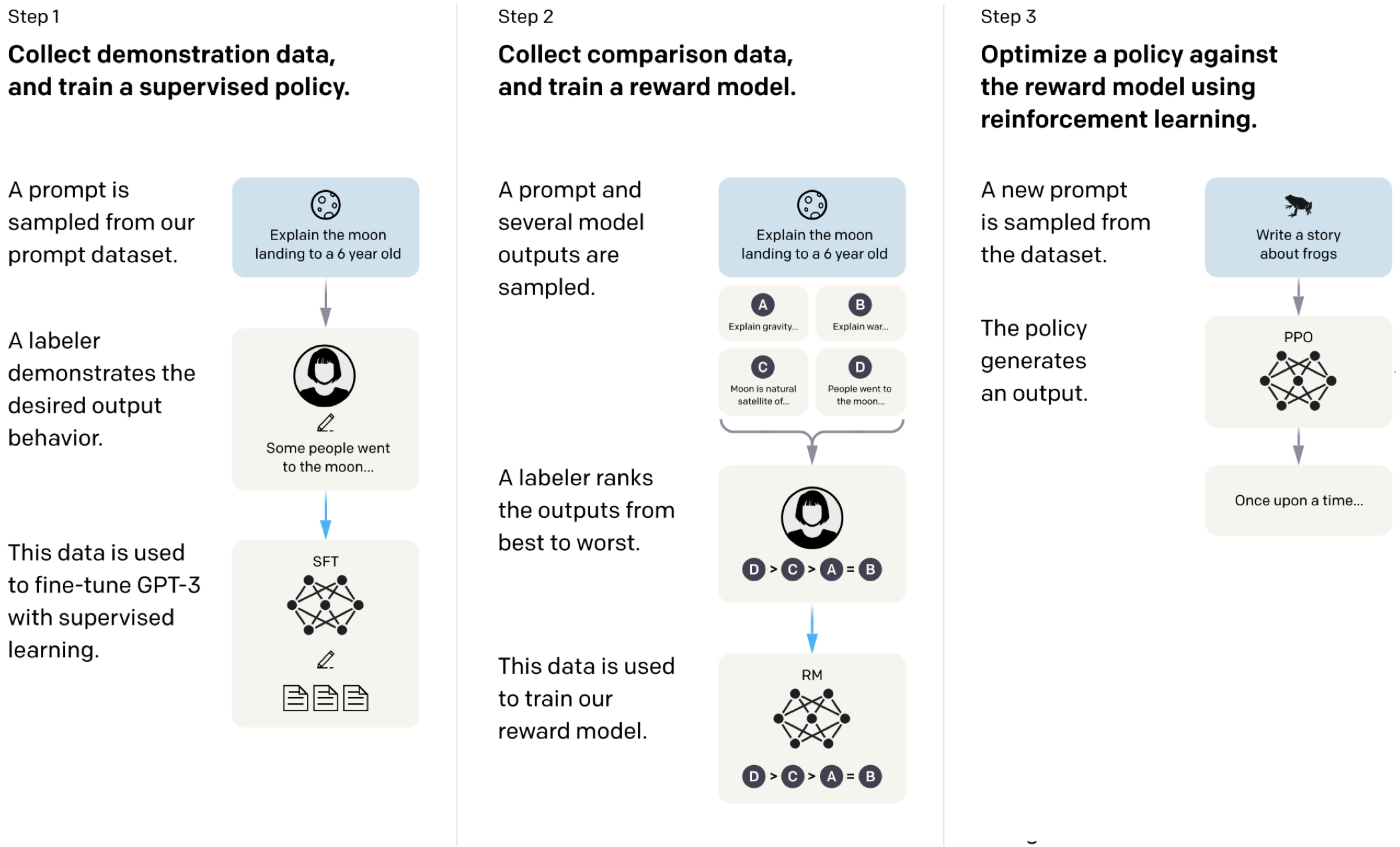
RLHF pipeline

Step 3: Reinforcement learning on the reward model



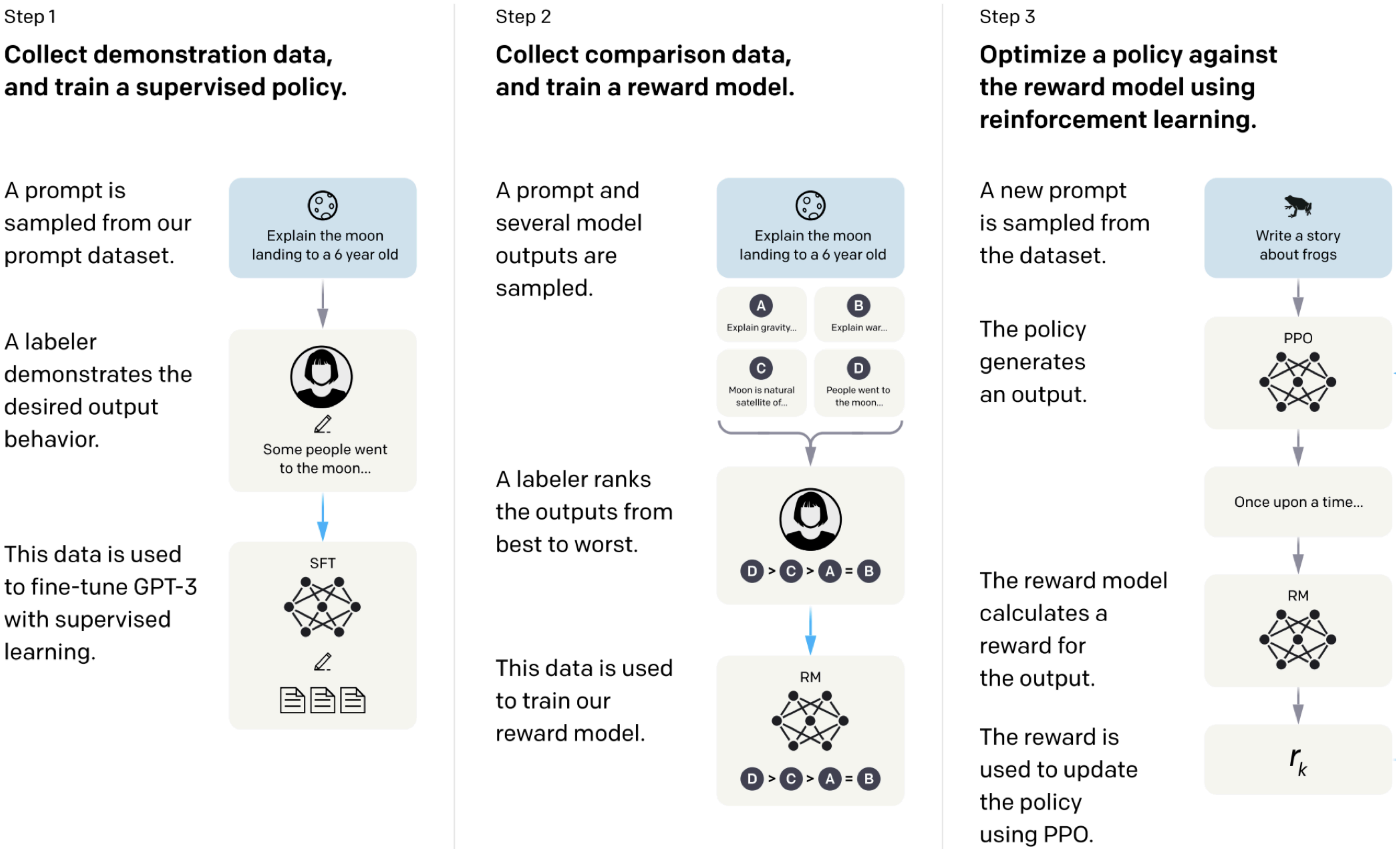
RLHF pipeline

Step 3: Reinforcement learning on the reward model



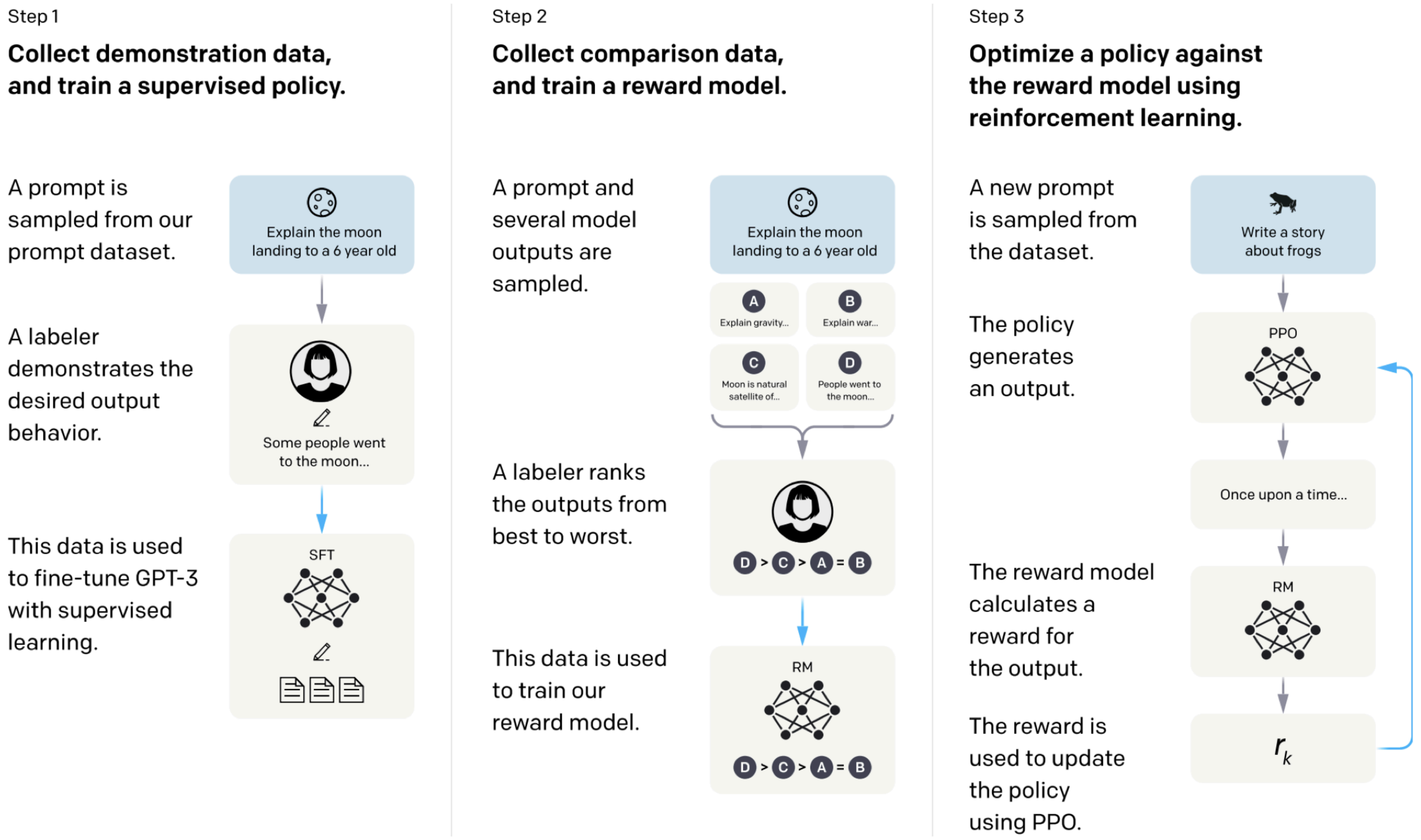
RLHF pipeline

Step 3: Reinforcement learning on the reward model



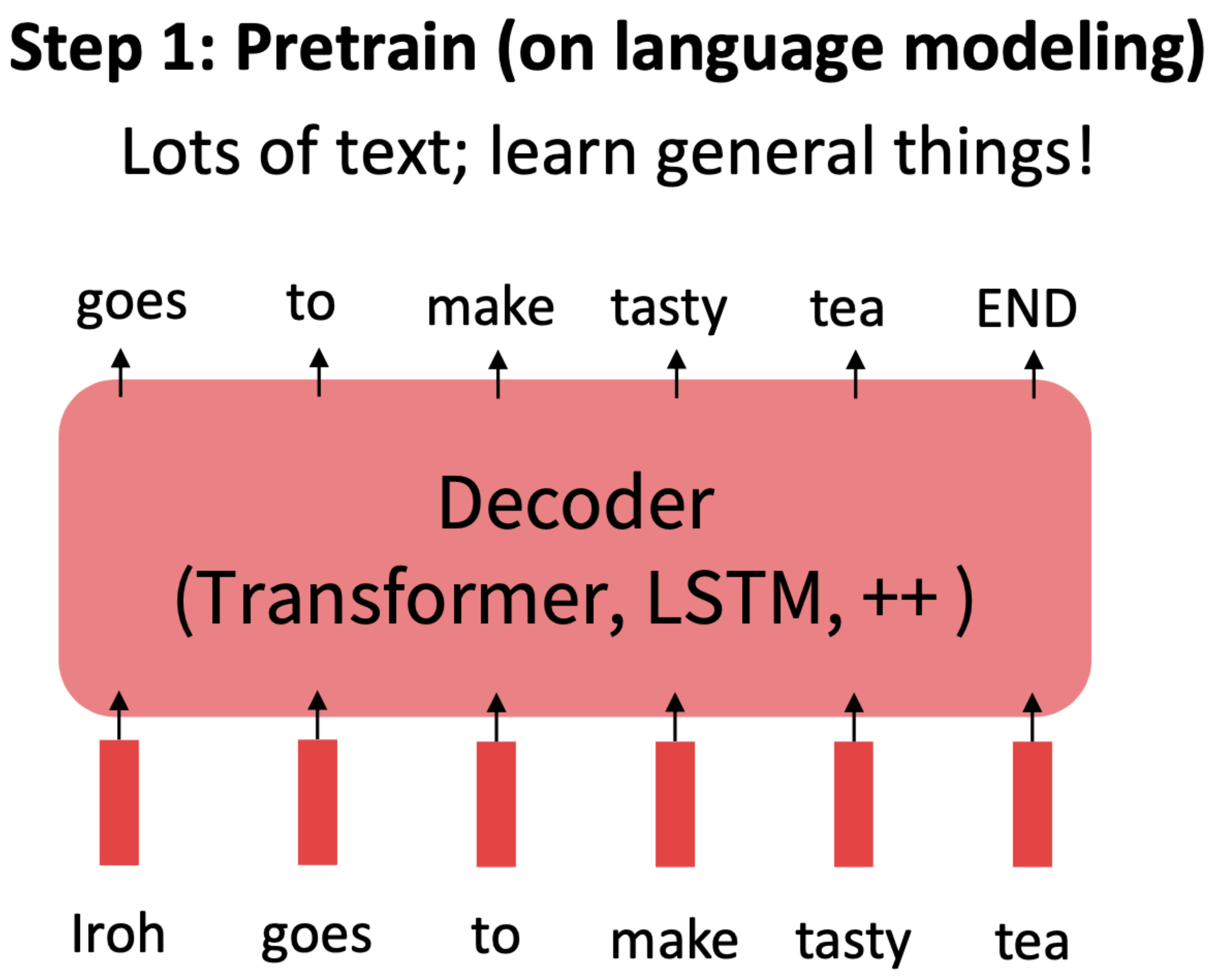
RLHF pipeline

Step 3: Reinforcement learning on the reward model



Starting point for RLHF

We start with **pre-trained base model**

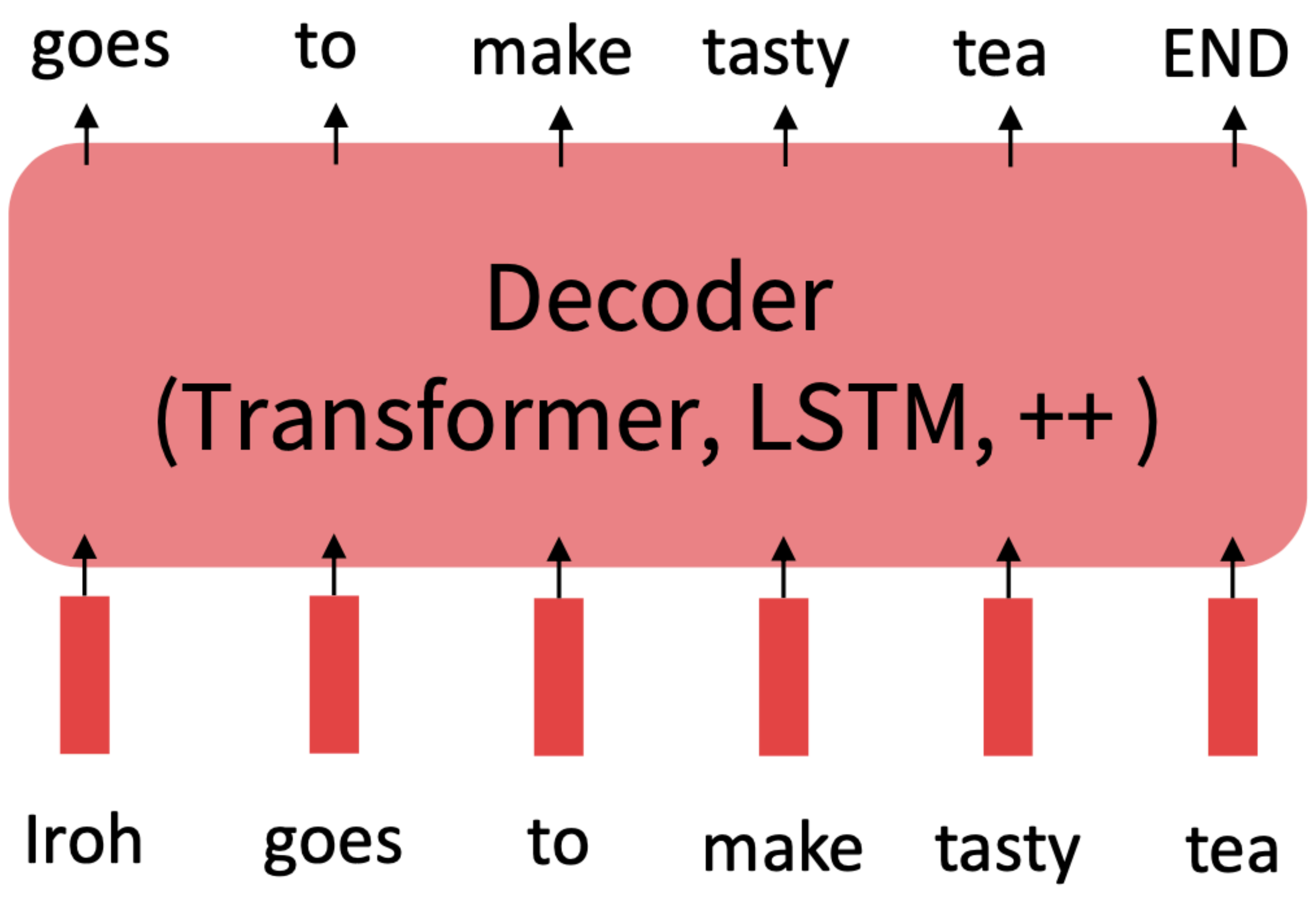


Starting point for RLHF

We start with **pre-trained base model** fine-tuned on **a downstream task**

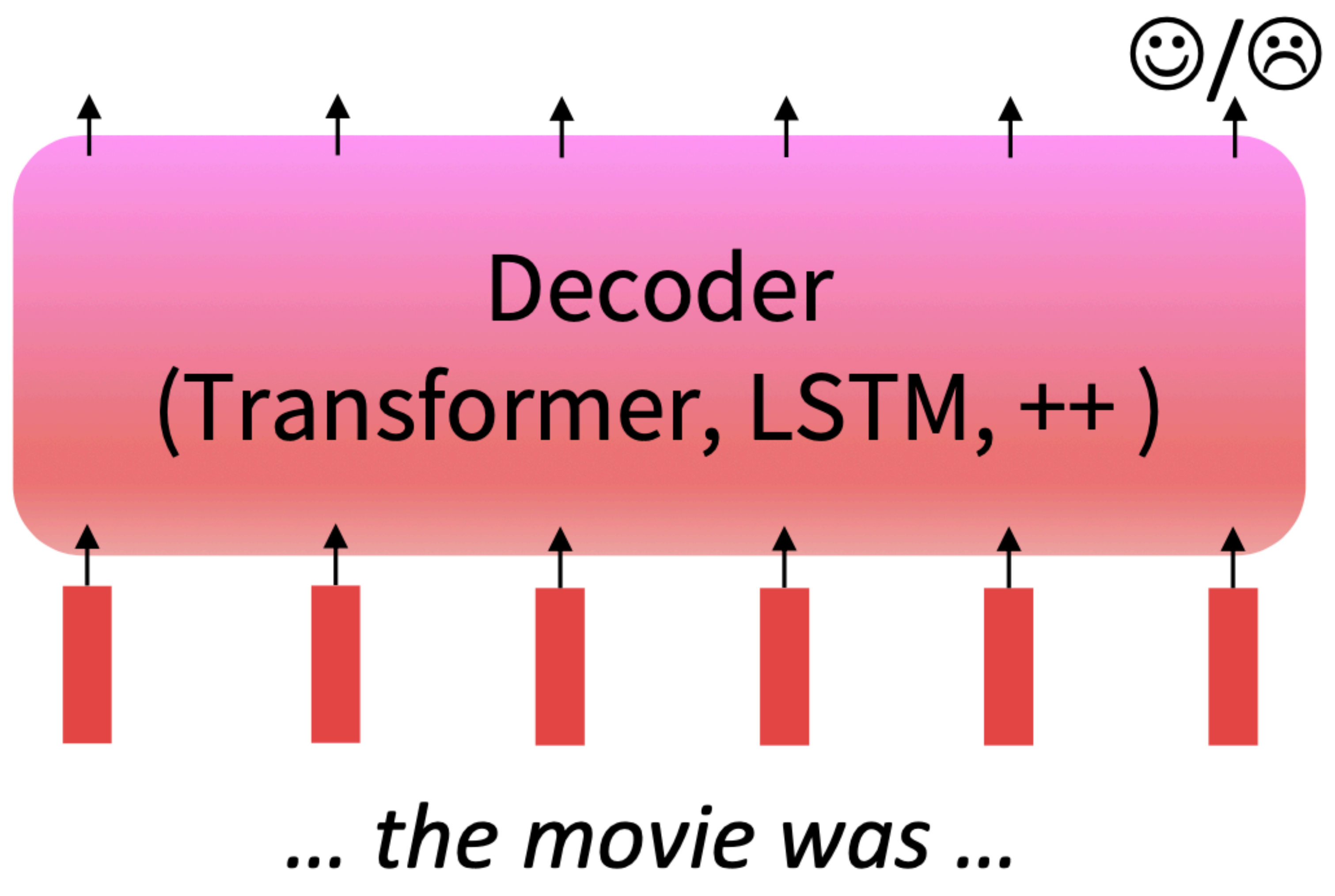
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!

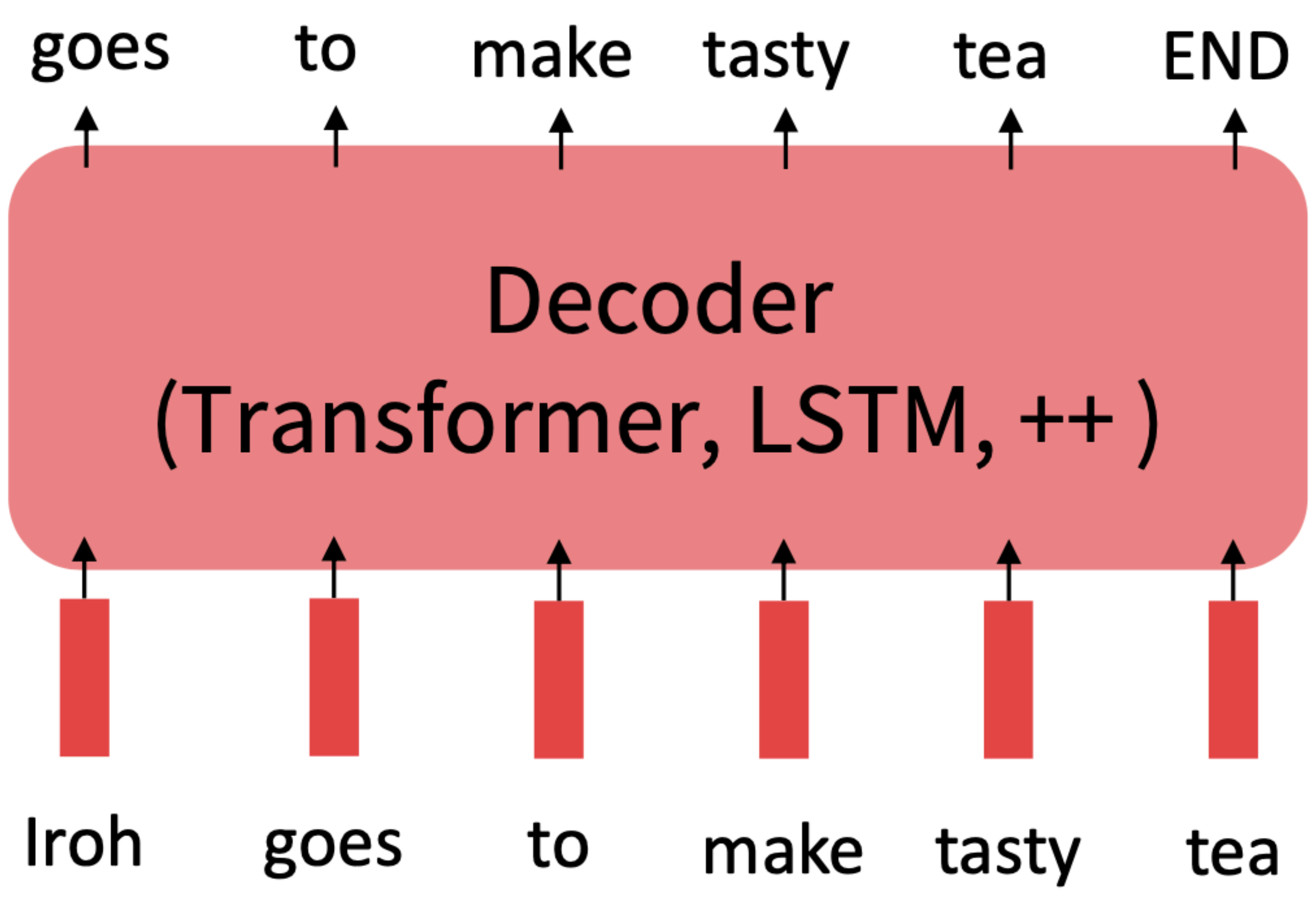


Instruction fine-tuning

We start with **pre-trained base model** fine-tined on **many downstream tasks**

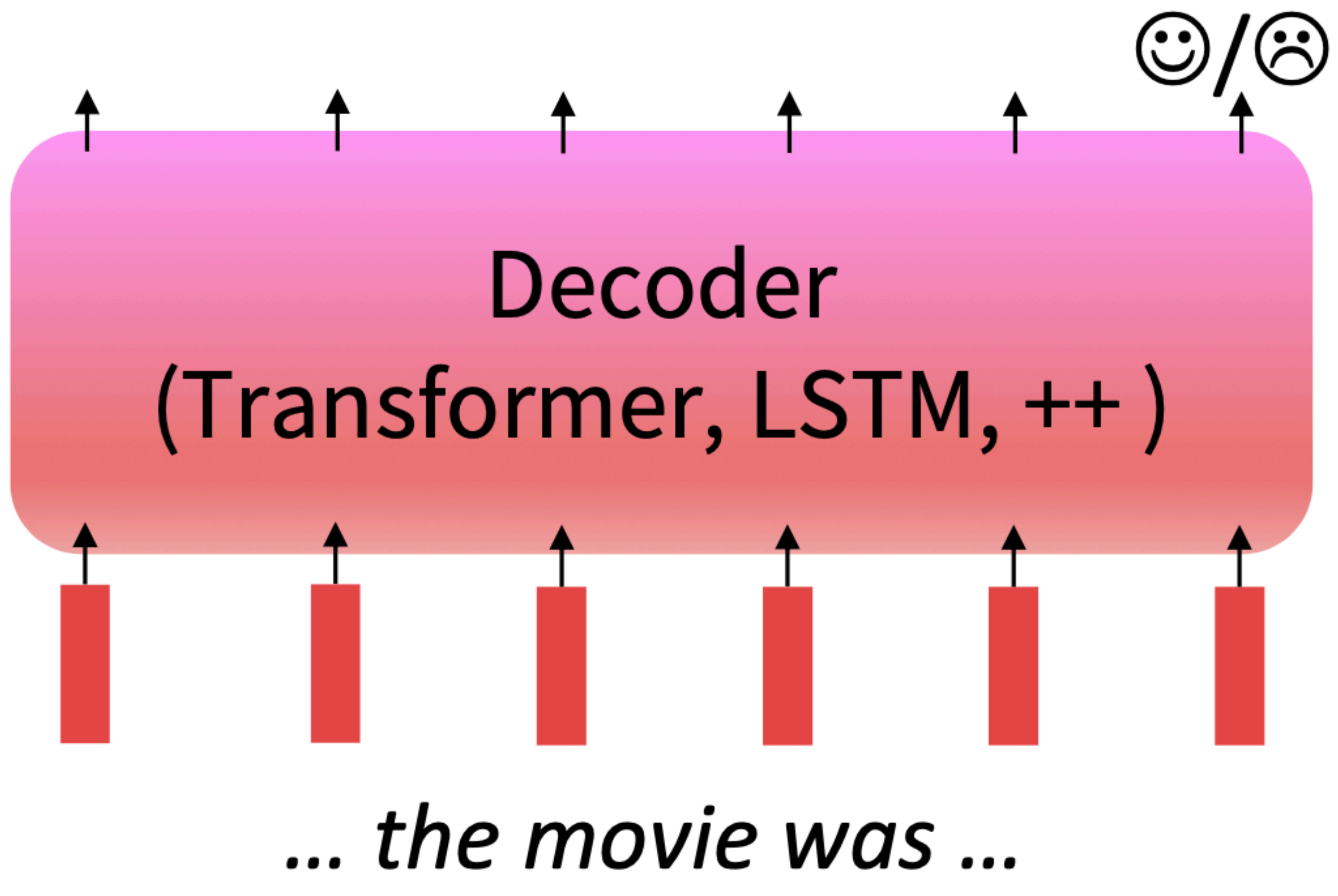
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on many tasks)

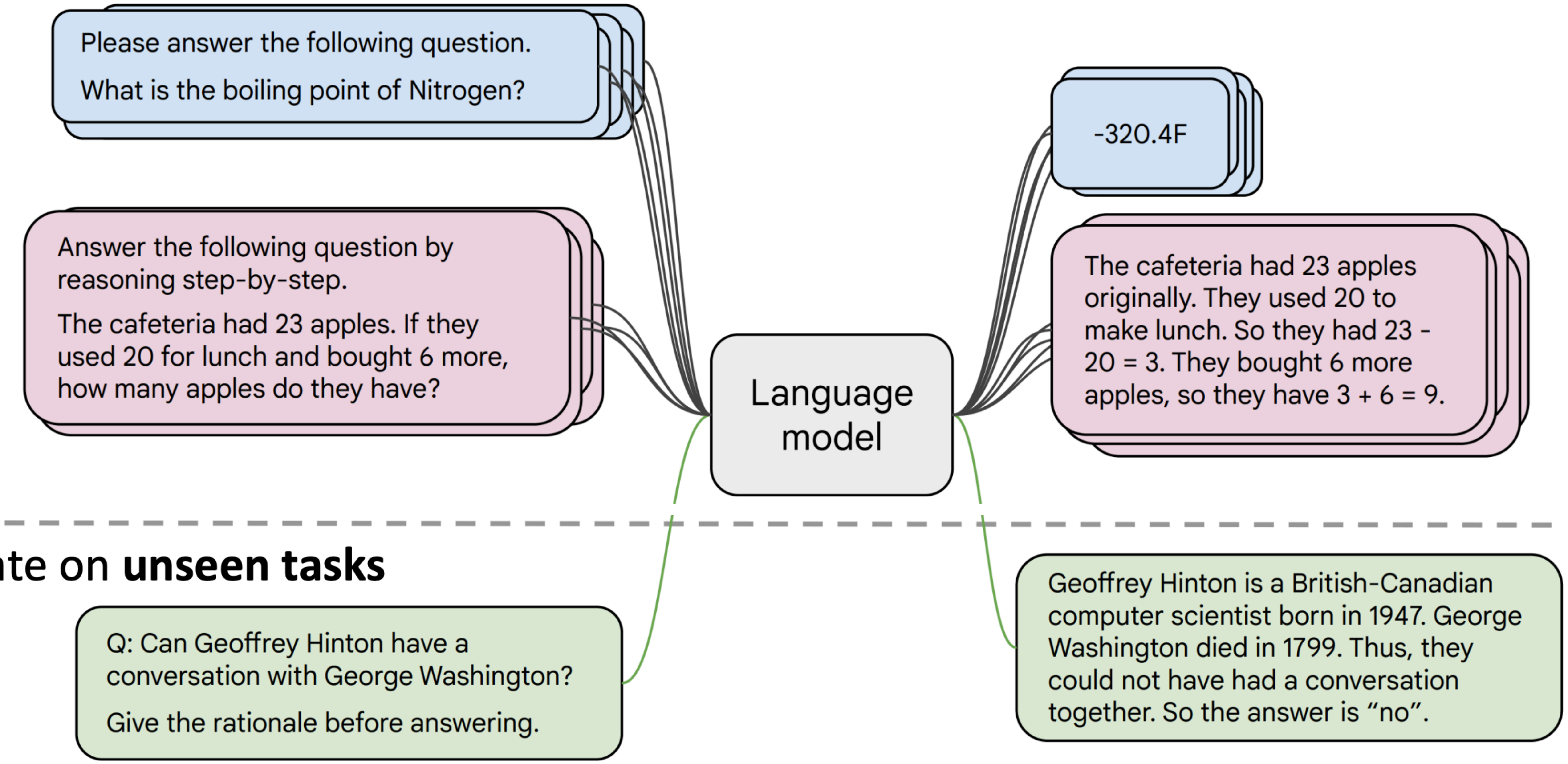
~~Not~~ many labels; adapt to the tasks!



Instruction fine-tuning

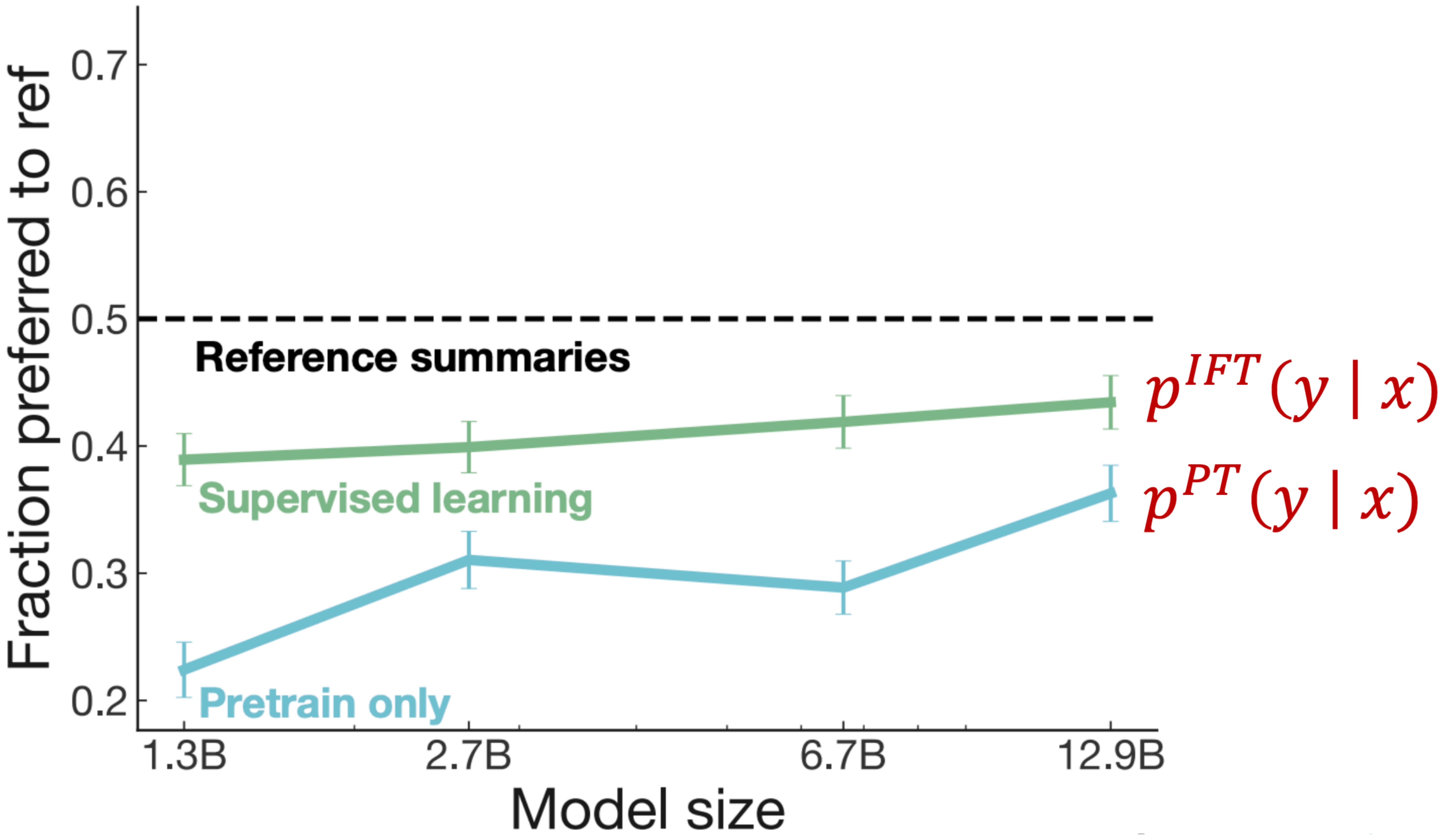
We start with **pre-trained base model** fine-tined on **many downstream tasks**

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM
- Evaluate on **unseen tasks**



Instruction fine-tuning

Instruction fine-tuned model often **does better** than than **pre-trained model**



Reward model

Consider the task of summarization. For instruction x and LM sample y, we obtain a human reward of that summary $R(x, y) \in \mathbb{R}$, higher is better

```
SAN FRANCISCO,
California (CNN) --
A magnitude 4.2
earthquake shook the
San Francisco
...
overturn unstable
objects.
```

x

```
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.
```

y_1

```
The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.
```

y_2

Reward model

Consider the task of summarization. For instruction x and LM sample y, we obtain a human reward of that summary $R(x, y) \in \mathbb{R}$, higher is better

SAN FRANCISCO, California (CNN) -- A magnitude 4.2 earthquake shook the San Francisco ... overturn unstable objects.	An earthquake hit San Francisco. There was minor property damage, but no injuries.	The Bay Area has good weather but is prone to earthquakes and wildfires.
x	y_1	y_2
	$R(x, y_1) =$	$R(x, y_2) =$

We want to maximize ____________

How do we get rewards?

Different labelers give different scores for the same response — leading to noisy data

```
A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.
```

$$y_3$$

$$R(x, y_3) = \quad 4.1? \quad 6.6? \quad 3.2?$$

How do we get rewards?

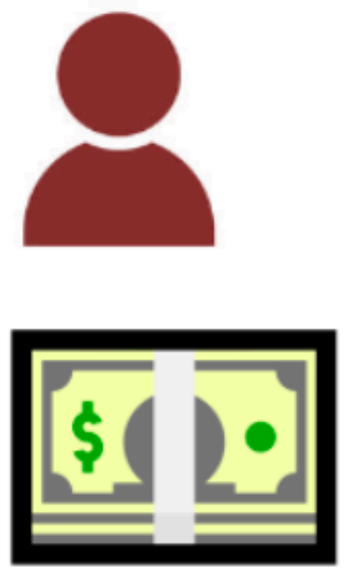
Modeling **human preferences is expensive.**

```
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.
```

```
The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.
```

$R(x, y_1) = 8.0$

$R(x, y_2) = 1.2$



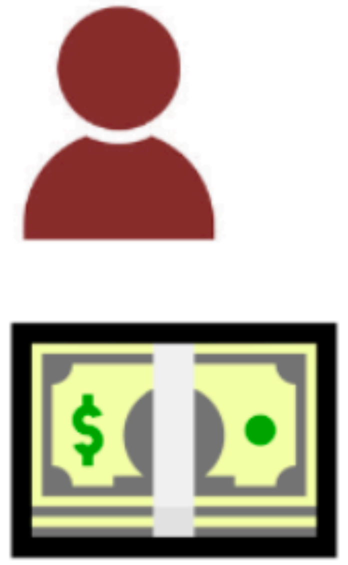
How do we get rewards?

Modeling **human preferences is expensive.**

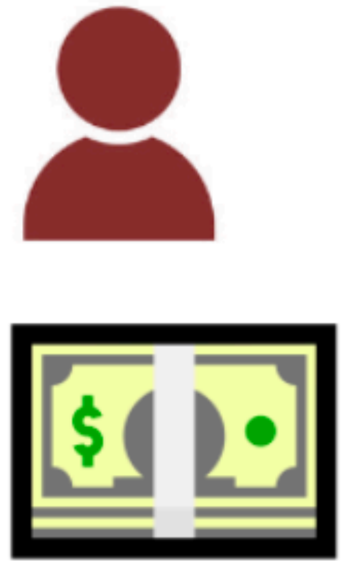
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

$$R(x, y_1) = 8.0$$



$$R(x, y_2) = 1.2$$



Train a model $\mathrm{RM}_\phi(x, y)$ that takes x as input to predict y to model human preferences

How do we get rewards?

Use pairwise comparisons — compare two responses and decide which one is better.

An earthquake hit San Francisco. There was minor property damage, but no injuries.	A 4.2 magnitude earthquake hit San Francisco, resulting in massive damage.	The Bay Area has good weather but is prone to earthquakes and wildfires.
y_1	y_3	y_2

How do we get rewards?

Use pairwise comparisons — compare two responses and decide which one is better.

```
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.
```

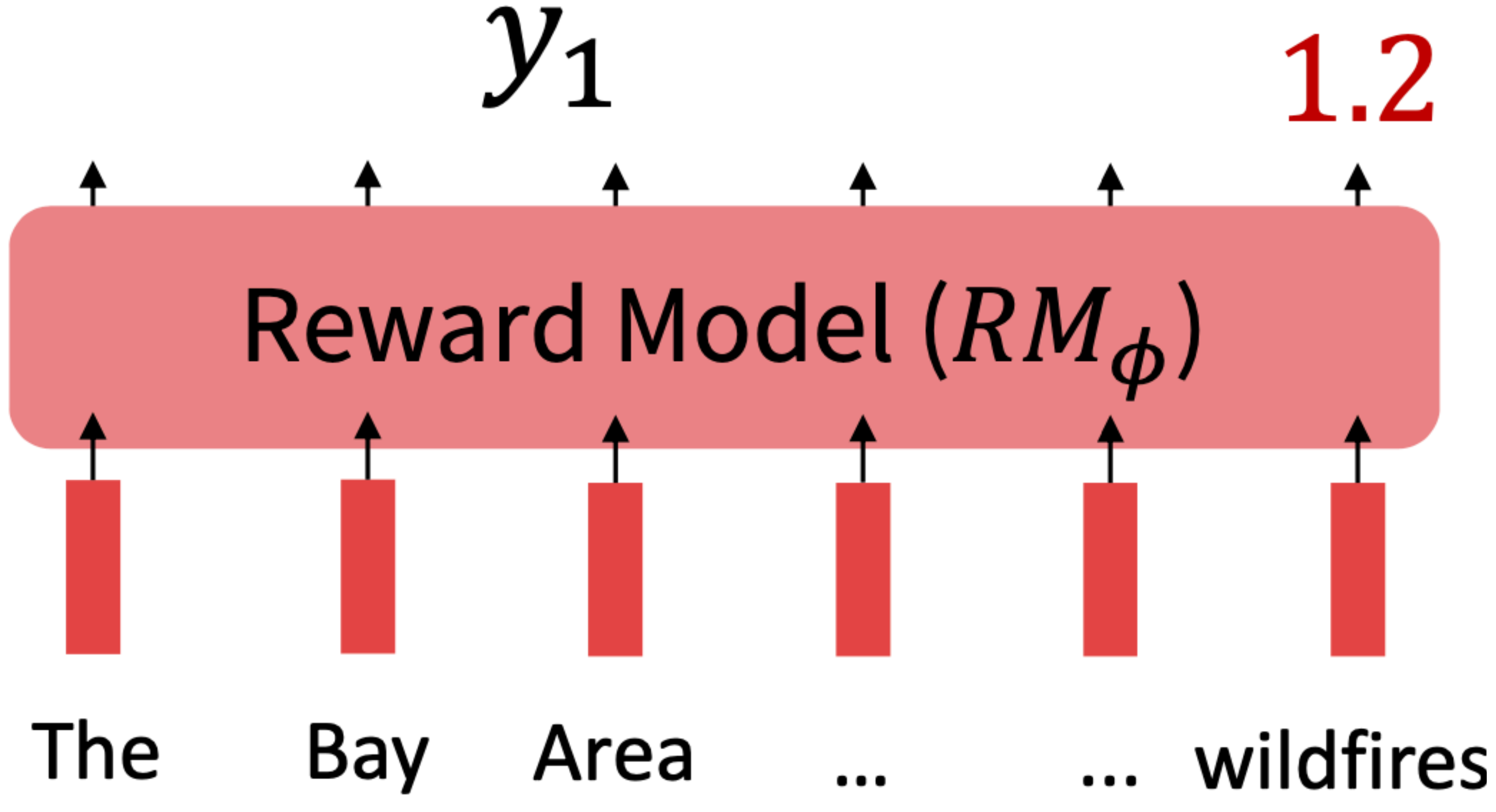
y_1

```
A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.
```

y_3

```
The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.
```

y_2



How do we get rewards?

Use pairwise comparisons — compare two responses and decide which one is better.

```
An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.
```

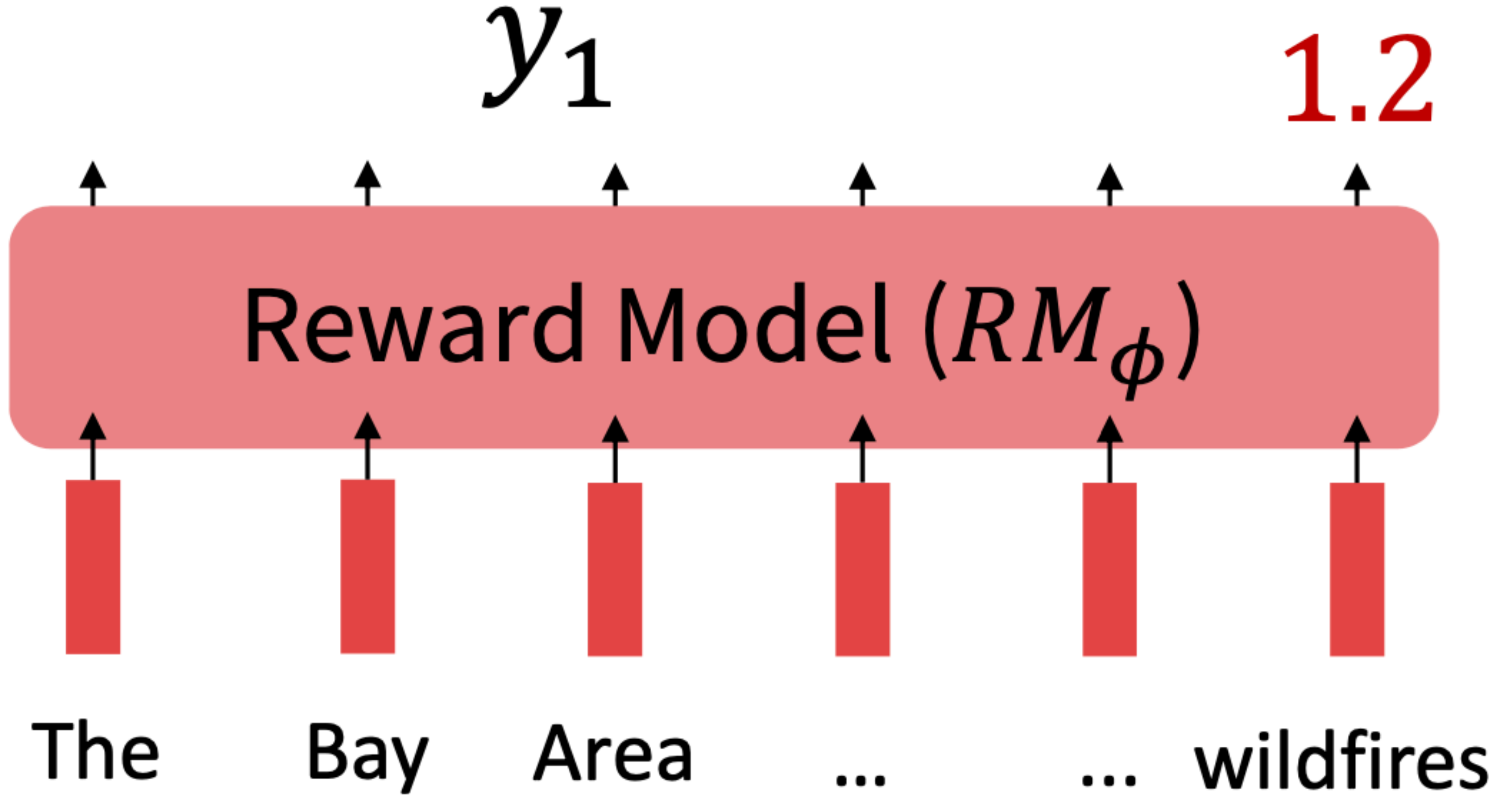
y_1

```
A 4.2 magnitude
earthquake hit
San Francisco,
resulting in
massive damage.
```

y_3

```
The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.
```

y_2



Bradley-Terry [1952] paired comparison model

$$J_{RM}(\phi) = -\mathbb{E}_{(x, y^w, y^l) \sim D}\left[\log \sigma(RM_\phi(x, y^w) - RM_\phi(x, y^l))\right]$$

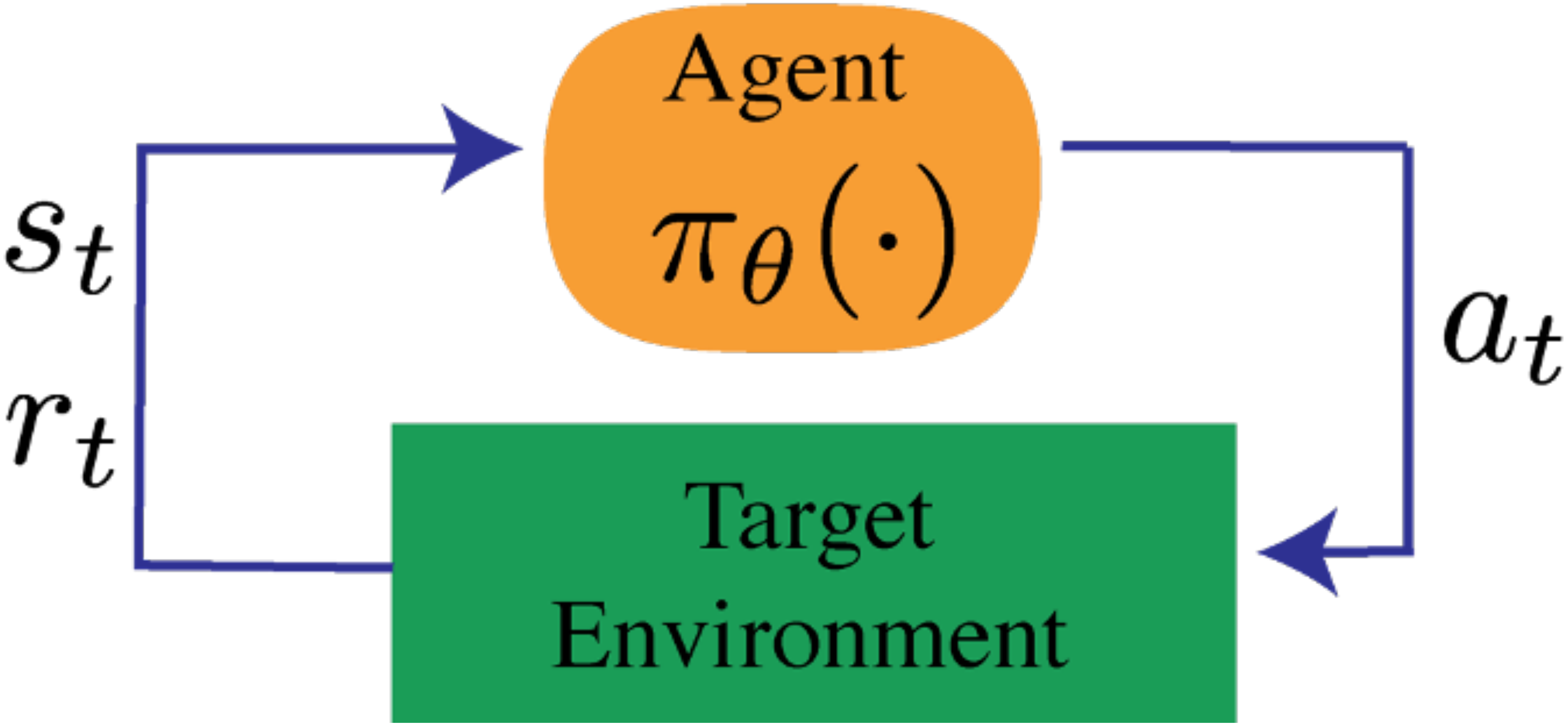
"winning" sample

"losing" sample

y^w should score higher than y^l

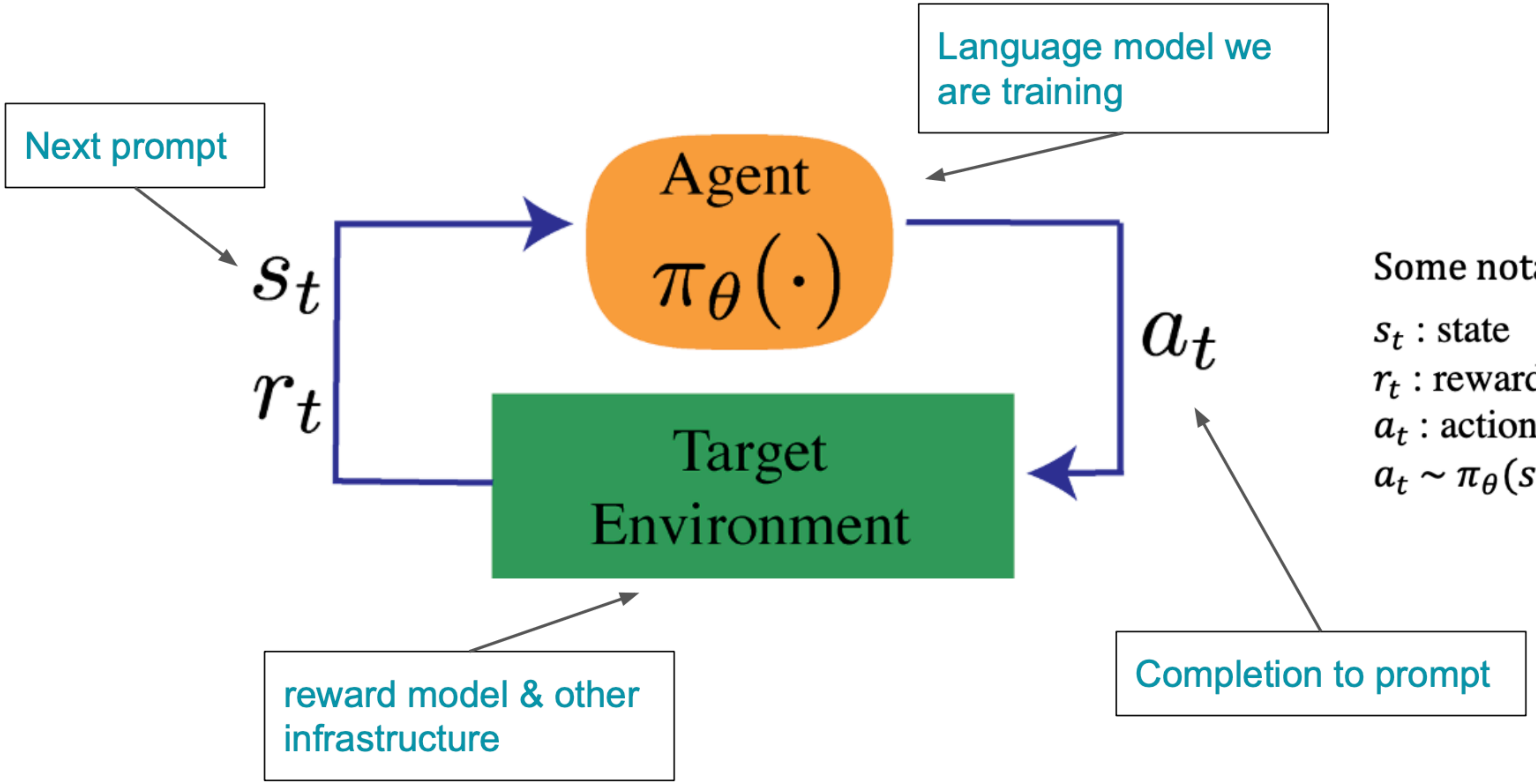
Background for Reinforcement Learning

Reinforcement learning is the problem of getting an agent to act in the world so as to maximize its rewards.



Background for Reinforcement Learning

Policy is a language model that takes in a prompt and returns a sequence of text

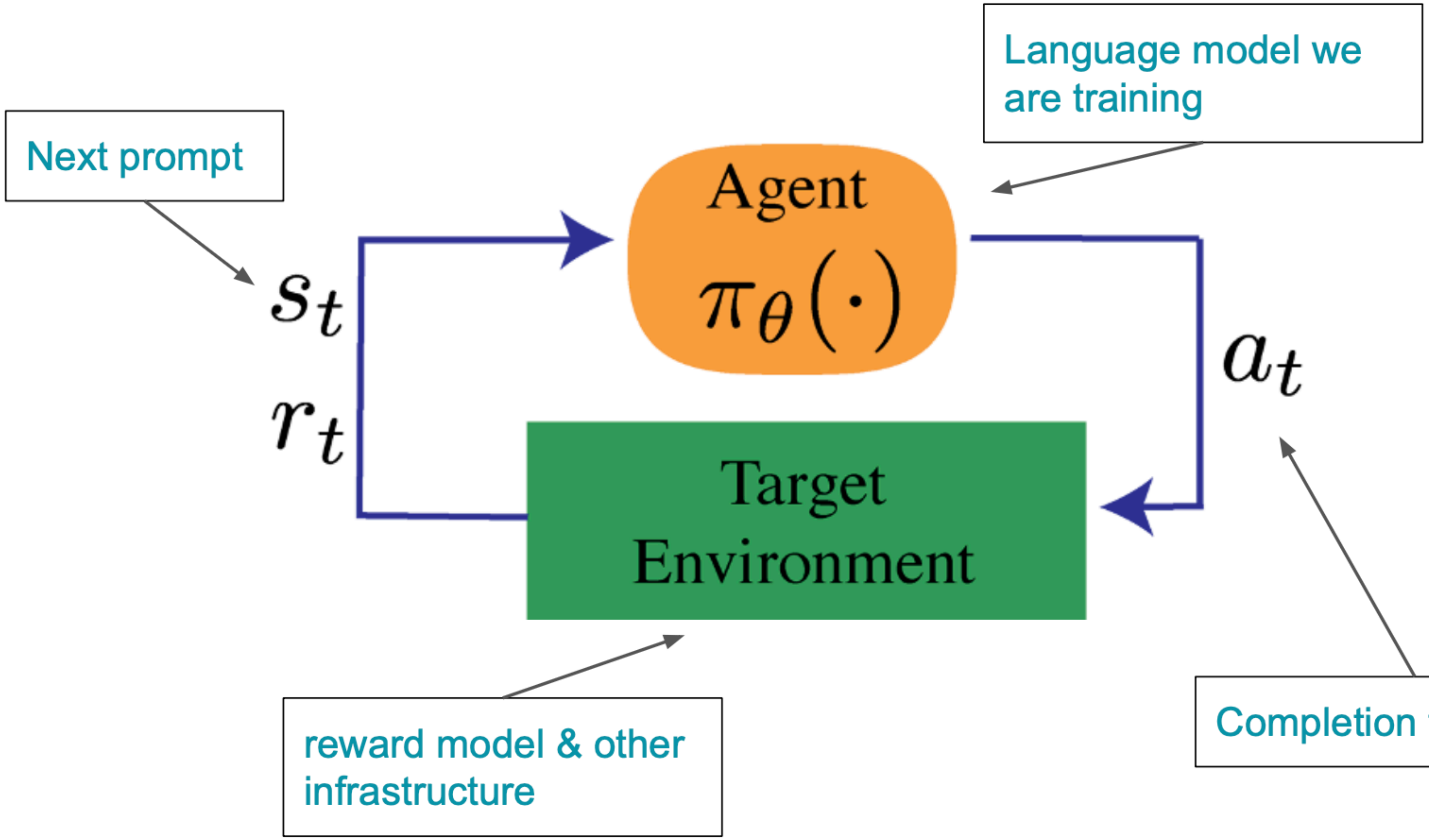


Some notation:

s_t : state
r_t : reward
a_t : action
$a_t \sim \pi_\theta(s_t)$: policy

Background for Reinforcement Learning

Action space is all the tokens in the vocabulary of the language model

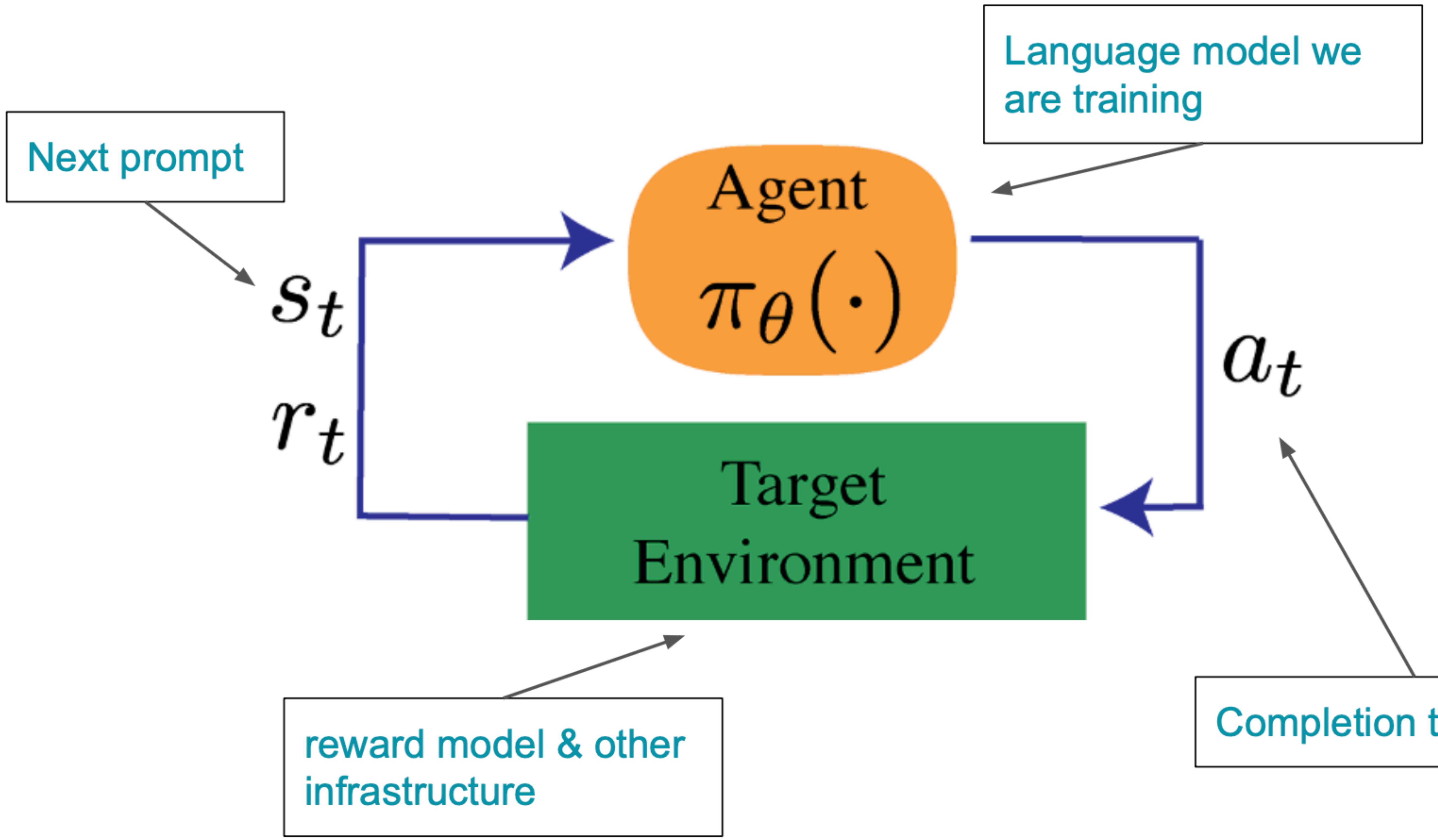


Some notation:

s_t : state
r_t : reward
a_t : action
$a_t \sim \pi_\theta(s_t)$: policy

Background for Reinforcement Learning

Reward function is a preference model and constraint on policy shift.



Some notation:

s_t : state
r_t : reward
a_t : action
$a_t \sim \pi_\theta(s_t)$: policy

Optimizing the reward model

Given a prompt, x from the dataset, the text $\hat{y}$ is generated by the current iteration of the fine-tuned policy. We optimize the following objective

$$\mathbb{E}_{\hat{y} \sim p_{\theta}^{RL}(\hat{y}|x)} \left[RM_{\phi}(x, \hat{y}) \right]$$

Optimizing the reward model

Given a prompt, x from the dataset, the text $\hat{y}$ is generated by the current iteration of the fine-tuned policy. We optimize the following objective

$$\mathbb{E}_{\hat{y} \sim p_\theta^{RL}(\hat{y}|x)} \left[RM_\phi(x, \hat{y}) \right]$$

Learned rewards ________ so the quantity can be imperfectly optimized

Optimizing the reward model

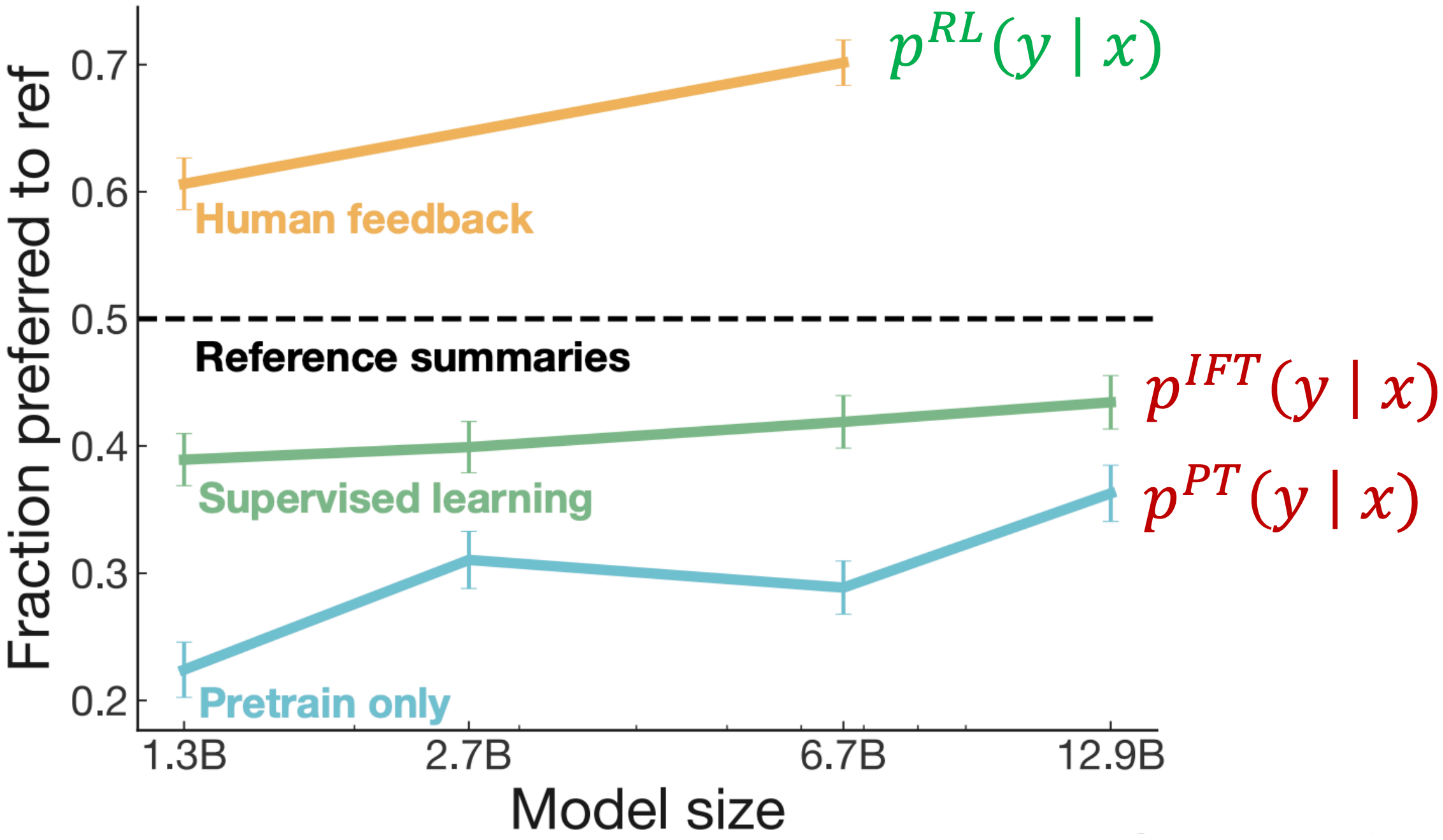
Add a penalty for drifting too far from the initialization

$$\mathbb{E}_{\hat{y} \sim p_\theta^{RL}(\hat{y}|x)} \left[RM_\phi(x, \hat{y}) - \beta \log \left(\frac{p_\theta^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)} \right) \right]$$

Penalty **prevents us from diverging** too far from the pre-trained model

Results with RLHF

RLHF **does better** than than both IFT and pre-trained model



Takeaways